Team rehearsal and presentation plan

Group 15 • Final assessment preparation • 5 November 2026

The group combines Computing Science, Data Science, and Business Informatics. Use these backgrounds as teaching strengths, then rotate explanations until each member can defend the whole project. The rubric explicitly permits redirected questions and different individual grades. This plan assigns preparation responsibilities, not historical authorship.

Shared readiness standard

Every member should independently draw the architecture, calculate a complete-sensor example, explain database/Kafka replay and the external duplicate window, read a raw benchmark, distinguish two bottlenecks, and locate the relevant implementation/test. Memorizing component names is insufficient if the member cannot predict behavior after an input or failure changes.

Preparation lead	Teach the others	Learn back from the others
Computing Science	Ordering, atomic commit, versions, cache races, backpressure	Statistical limits, workload accounting, cost and business risk
Data Science	Offered versus achieved rate, maxima/percentiles, random traces, confounders, capacity experiments	Checkpoint/outbox and cache coherence from actual code
Business Informatics	Requirement traceability, value, tradeoffs, cost scope, incident decisions	Complete grouping, concurrency, and performance arithmetic

Scientific understanding here includes experiment design and evidence quality; the assignment does not require machine learning. Business analysis includes precise service contracts and operational tradeoffs; it is not restricted to presenting the introduction.

Four rehearsal sessions

Session 1, 45 minutes: derive the behavior. Read the submission report and draw the diagram independently. Each member calculates exercise A in the handbook, then introduces a missing sensor, duplicate, and boundary value. Finish by explaining why three outside points can require only two alerts.

Session 2, 60 minutes: defend the implementation. Open `engine.py`, `executor.py`, `data.py`, `scheduler.py`, and `notification.py`. Rotate the person explaining each function. Place a crash before commit, after commit, and after remote acceptance. Then trace the delayed-writer metadata race and the stale-middle-version test. Every explanation must identify an invariant and a test.

Session 3, 45 minutes: challenge the evidence. One person states a claim, another finds its JSON values, and the third challenges the inference. Rotate using 703/703, 202 comparisons, 190.573/s, 5,526/6,000 offers, and the independent failed recovery phase. Ask why the native variable run's configured maximum is not an observed 200 phase and why final scheduler changes need separate evidence.

Session 4, 40 minutes: mock assessment. Deliver the ten-minute practice presentation below, then spend twenty minutes on random redirection and ten minutes correcting weak answers. One member answers, one acts as examiner, and one checks mechanism/evidence; rotate after every question. Ten minutes is a rehearsal format, not a confirmed exam allocation.

If only one hour remains: ten minutes quick reference, twenty minutes architecture/crash examples, twenty minutes random questions, and ten minutes demo/evidence navigation. This is a fallback study plan, not equivalent to the full rehearsal.

Ten-minute presentation

Time	Rehearsal lead	Message and artifact
0:00–1:00	Business Informatics	Monitoring-only problem, complete-sensor mean, traceability and transition alerts
1:00–2:30	Computing Science	Architecture diagram, responsibilities, credible monolith alternative
2:30–4:00	Computing Science	One transaction and two crash boundaries; correctness before optimization
4:00–6:00	Data Science	Native variable/fixed results and independent failed boundary; exact workload and counts
6:00–7:30	Data Science	Database/read/controller bottlenecks and targeted revisions; limits of causal claims
7:30–9:00	Business Informatics	Disk incident, live recovery, cost assumption, remaining failure domain
9:00–10:00	Any member	Show current readiness and state the next discriminating experiment

Use the architecture, failure-boundary, HTTP-phase, and replica figures supplied with the pack. Keep one raw result and one source function ready to open. The complete code appendix is a reference for questions, not something to scroll through during the main explanation.

Suggested opening: “We preserve complete-sensor experiment history and notify the required temperature transitions. We separated ordered intake, computation, durable storage, queries, and delivery. The core guarantee is that state, history, alert obligations, and input progress commit together. We measured the system, found avoidable database and coordination work, and improved it while retaining the consistency checks.”

Suggested result statement: “Our native variable run checked five observed HTTP targets with 100 experiments, ten sensors, and one-second samples. All 703 required alerts arrived, maximum 2.884 seconds, and both full histories matched for 101 experiments. The separate 200 target passed narrowly, while an independent live-history 200 peak still failed. We retain both results because workload definition matters.”

Suggested limitation statement: “Responsibilities can scale separately, but the deployment shares one VM and Data's invalidation currently assumes one writer process. A hot partition, a saturated database, and a slow notification receiver need different remedies. Our next performance evidence should repeat representative runs on the exact final scheduler revision and then vary the identified limiting factor.”

Redirection and challenge cards

- Ask Business Informatics to explain the delayed-writer cache race and what the generation check prevents.
- Ask Data Science to trace durable offset 45 versus Kafka next-read offset 46.
- Ask Computing Science why before/after maxima do not isolate causal improvement.
- Ask Business Informatics why 5,526 error-free responses can fail a 6,000-offer phase.
- Ask Data Science why clearing Data's cache matters in a PostgreSQL recovery test.
- Ask Computing Science to distinguish provisioned storage cost from database-used bytes.
- Ask any member whether a zero scheduler queue proves zero broker lag.
- Ask any member why the external hostname exception is a compromise.
- Ask any member whether reducing a fifteen-second lease guarantees the notification SLA.

- Ask any member which test actually measures the final scheduler revision's capacity.

For each answer the observer records the incorrect assumption, source/evidence that resolves it, and a new example to retry five minutes later. Reward a precise correction, not confidence without support.

Score understanding without inventing grades

Use 0 = cannot explain, 1 = terms without application, 2 = correct explanation and example, 3 = mechanism plus evidence, tradeoff, and counterexample. These are study scores, not a mapping to the professor's marks. Aim for every member at least 2 on every row.

Topic	Computing Science	Data Science	Business Informatics
Domain and interfaces	Unassessed	Unassessed	Unassessed
Partition order, versions, replay	Unassessed	Unassessed	Unassessed
Outbox and external duplicate window	Unassessed	Unassessed	Unassessed
Snapshot and metadata coherence	Unassessed	Unassessed	Unassessed
Workloads, arithmetic, experimental limits	Unassessed	Unassessed	Unassessed
Bottleneck diagnosis and scaling	Unassessed	Unassessed	Unassessed
Demo modes, costs, and recovery	Unassessed	Unassessed	Unassessed

Actual contribution record

Complete this from real work before using it in a submission. Include implementation, experiment execution, analysis, reviews, operations, documentation, or teaching that actually occurred. Link commits/files/results or dated review notes. Do not assign authorship from the preparation leads above.

Member	Actual work and decision	Artifact and verification
Computing Science member	To be completed by the member	To be linked to evidence
Data Science member	To be completed by the member	To be linked to evidence
Business Informatics member	To be completed by the member	To be linked to evidence

When uncertain in the exam, state the assumption, reason from state ownership and the failure boundary, then identify what you would inspect. A specific evidence limit is a stronger answer than inventing an observation. Follow the course's applicable assistance/disclosure policy when describing how the work was produced.